

Lab Report 9 - Cybersecurity Analytics

Machine Learning Model for Detecting Cyber Attacks in a Network

Introduction

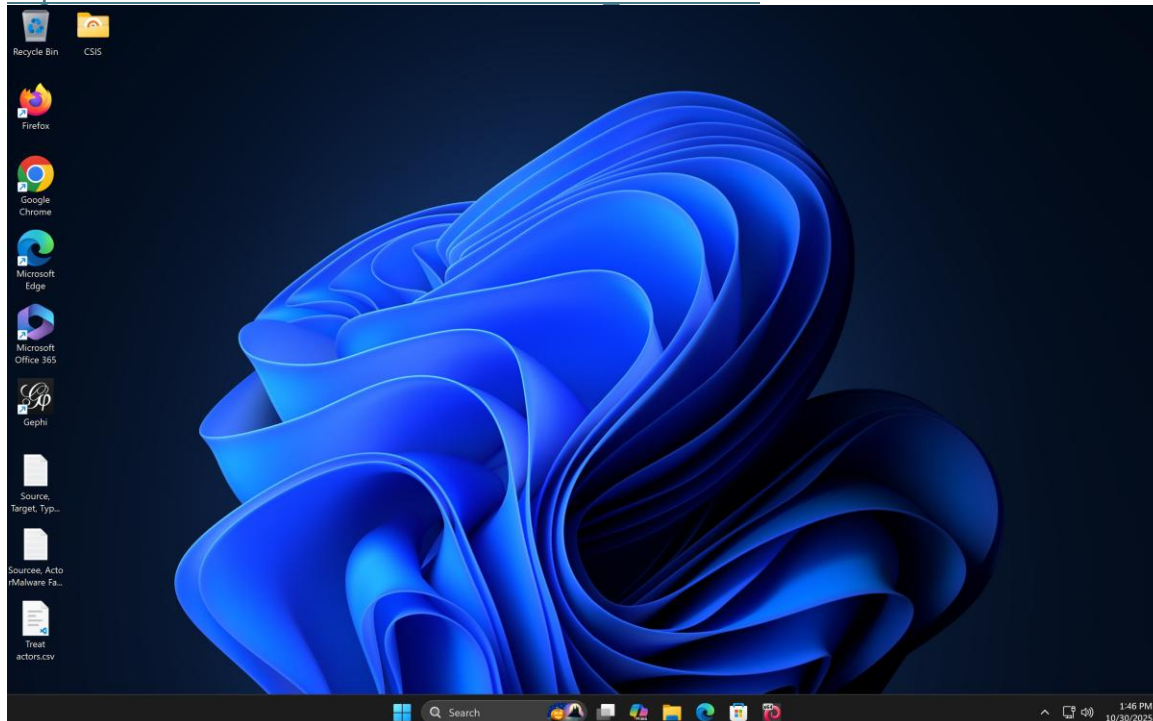
This lab is designed to work with CSIS-Admin VM machine to work with machine learning to detect, classify, and respond to network attacks. This lab works with two classification models called decision tree and a k-nearest neighbor (KNN) classifier to detect attack categories Benign, DoS (denial of service), R2L (remote-to-local), probe, and U2R (user-to-root) attacks. Choosing a suitable classifier for the task can be one of the more challenging parts of machine learning (Chio & Freeman, 2018).

This lab provides hands-on experience with how machine-learning systems are built and designed to security domains where imbalanced classes, noisy data, and evolving attack strategies complicate model design. We can calculate error rate and accuracy of each model to identify which model is most suitable to detect network attacks.

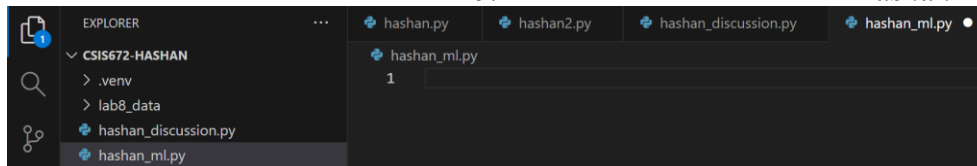
Part 1 - Decision Tree Classifier

1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.

https://labs.azure.com/virtualmachines?feature_vnext=true



2. Created new file under CSIS672-HASHAN and named as *Hashan ml.py*



3. Import the necessary packages and then loaded and stored data.

```

hashan_ml.py > ...
1 # Import the necessary packages
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.compose import ColumnTransformer, make_column_selector
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import confusion_matrix, zero_one_loss
9
10 # Load the Data
11 hashan_data_all = pd.read_csv("C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/lab_data/lab_data_kdd_c.csv")
12

```

4. Found that scikit-learn library file cannot find in VM. Then installed the library file.

```

(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-1.7.2-cp313-cp313-win_amd64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.22.0 in c:\users\csis-admin\documents\csis672-hashan\.venv\lib\site-packages (from scikit-learn) (2.3.4)
Collecting scipy>=1.8.0 (from scikit-learn)
  Downloading scipy-1.16.3-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.2-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.7.2-cp313-cp313-win_amd64.whl (8.7 MB)
   8.7/8.7 MB 71.5 MB/s 0:00:00
Downloading joblib-1.5.2-py3-none-any.whl (308 kB)
Downloading scipy-1.16.3-cp313-cp313-win_amd64.whl (38.5 MB)
   38.5/38.5 MB 49.4 MB/s 0:00:00
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-1.5.2 scikit-learn-1.7.2 scipy-1.16.3 threadpoolctl-3.6.0
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>

```

5. Printed the head of the data.

```

hashan_ml.py > ...
1 # Import the necessary packages
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.compose import ColumnTransformer, make_column_selector
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import confusion_matrix, zero_one_loss
9
10 # Load the Data
11 hashan_data_all = pd.read_csv("C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/lab_data/lab_data_kdd_c.csv")
12
13 #Print the head of the data
14 print(hashan_data_all.head())
15

```

	protocol_type	service	flag	land	...	dst_host_srv_error_rate	dst_host_error_rate	dst_host_srv_error_rate	category
0	tcp	ftp_data	SF	0	...	0.00	0.05	0.00	benign
1	udp	other	SF	0	...	0.00	0.00	0.00	benign
2	tcp	private	S0	0	...	1.00	0.00	0.00	dos
3	tcp	http	SF	0	...	0.01	0.00	0.01	benign
4	tcp	http	SF	0	...	0.00	0.00	0.00	benign

[5 rows x 42 columns]

```

(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>

```

6. The loaded data contains both predictor features, X and target feature, Y. X features are used to predict an attack category, Y. Need to separate the X features, the from Y to prepare the data for creating a classification model. In the loaded dataset, select all columns but the last column. The last column contains the target, y

$$x_total = hashan_data_all.iloc[:, :-1]$$

Values for Y features, that is, the target feature, are stored in the column labeled category. The following command is used to separate the Y features.

y_total = hashan_data_all['category']

```
16 #Separate Predictors and Target
17 x_total = hashan_data_all.iloc[:, :-1]
18 y_total = hashan_data_all['category']
```

7. Scaled the continuous features and encoded the nominal features using following commands.

```
20 #Identify and separate Feature Types
21 cont_features = ['duration', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',
22                 'num_compromised', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files',
23                 'num_outbound_cmds', 'count', 'srv_count', 'error_rate', 'srv_error_rate', 'rerror_rate', 'srv_rerror_rate',
24                 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
25                 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
26                 'dst_host_error_rate', 'dst_host_srv_error_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate']
27
28 binary_features = ['land', 'logged_in', 'root_shell', 'su_attempted', 'is_host_login', 'is_guest_login']
29 nominal_features = ['protocol_type', 'service', 'flag']
```

8. Converted nominal features into indicator features using following command.

```
31 #Convert nominal features into indicator features
32 all_x_data = pd.get_dummies(x_total, columns= nominal_features, drop_first=True)
```

9. Checked the outcome of the conversion by printing the first few rows of the data.

```
hashan_ml.py X
hashan_ml.py > ...
1 # Import the necessary packages
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.compose import ColumnTransformer, make_column_selector
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import confusion_matrix, zero_one_loss
9
10 # Load the Data
11 hashan_data_all = pd.read_csv("C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/lab_data/lab_data_kdd_c.csv")
12
13 #Print the head of the data
14 print(hashan_data_all.head())
15
16 #Separate Predictors and Target
17 x_total = hashan_data_all.iloc[:, :-1]
18 y_total = hashan_data_all['category']
19
20 #Identify and separate Feature Types
21 cont_features = ['duration', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',
22                 'num_compromised', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files',
23                 'num_outbound_cmds', 'count', 'srv_count', 'error_rate', 'srv_error_rate', 'rerror_rate', 'srv_rerror_rate',
24                 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
25                 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
26                 'dst_host_error_rate', 'dst_host_srv_error_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate']
27
28 binary_features = ['land', 'logged_in', 'root_shell', 'su_attempted', 'is_host_login', 'is_guest_login']
29 nominal_features = ['protocol_type', 'service', 'flag']
30
31 #Convert nominal features into indicator features
32 all_x_data = pd.get_dummies(x_total, columns= nominal_features, drop_first=True)
33
34 #Check the outcome of the conversion
35 print(all_x_data[0:3])
```

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [x] [x] [x] [x]
```

```
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> & C:\Users\CSIS-Admin\Documents\CSIS672-Hashan\.venv\Scripts\python.exe c:/Users/CSIS-Admin/Documents/CSIS672-Hashan/hashan_ml.py
```

	protocol_type	service	flag	land	logged_in	...	dst_host_error_rate	dst_host_srv_error_rate	dst_host_errror_rate	dst_host_srv_rerror_rate	category
0	tcp	ftp_data	SF	0	0	...	0.00	0.00	0.05	0.00	benign
0	tcp	ftp_data	SF	0	0	...	0.00	0.00	0.05	0.00	benign
1	udp	other	SF	0	0	...	0.00	0.00	0.00	0.00	benign
2	tcp	private	S0	0	0	...	1.00	1.00	0.00	0.00	dos
3	tcp	http	SF	0	1	...	0.03	0.01	0.00	0.01	benign
4	tcp	http	SF	0	1	...	0.00	0.00	0.00	0.00	benign

[5 rows x 42 columns]

	land	logged_in	is_host_login	is_guest_login	duration	src_bytes	dst_bytes	...	flag_RSTF	flag_S0	flag_S1	flag_S2	flag_S3	flag_SF	flag_SH
0	0	0	0	0	0	491	0	...	False	False	False	False	False	True	False
1	0	0	0	0	0	146	0	...	False	False	False	False	False	True	False
2	0	0	0	0	0	0	0	...	False	True	False	False	False	False	False

[3 rows x 119 columns]

```
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>
```

10. Separated the predictor features, X, into training and test datasets using following commands.

```
x_train = all_x_data.iloc[0:125973]
```

```
x_test = all_x_data.iloc[125974:148517]
```

11. Ran the describe() function on x_train and x_test

```

1 # Import the necessary packages
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.compose import ColumnTransformer, make_column_selector
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import confusion_matrix, zero_one_loss
9
10 # Load the Data
11 hashan_data_all = pd.read_csv("C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/lab_data/lab_data_kdd_c.csv")
12
13 #Print the head of the data
14 print(hashan_data_all.head())
15
16 #Separate Predictors and Target
17 x_total = hashan_data_all.iloc[:, :-1]
18 y_total = hashan_data_all ['category']
19
20 #Identify and seperate Feature Types
21 cont_features = ['duration', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',
22                 'num_compromised', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files',
23                 'num_outbound_cmds', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate', 'error_rate', 'srv_error_rate',
24                 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
25                 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
26                 'dst_host_serror_rate', 'dst_host_srv_serror_rate', 'dst_host_error_rate', 'dst_host_srv_error_rate']
27
28 binary_features = ['land', 'logged_in', 'root_shell', 'su_attempted', 'is_host_login', 'is_guest_login']
29 nominal_features = ['protocol_type', 'service', 'flag']
30
31 #Convert nominal features into indicator features
32 all_x_data = pd.get_dummies(x_total, columns= nominal_features, drop_first=True)
33
34 #Check the outcome of the conversion
35 print(all_x_data[0:3])
36
37 #Separate Dataset into Training and Testing
38 x_train = all_x_data.iloc[0:125973]
39 x_test = all_x_data.iloc[125974:148517]
40
41 #Run the describe() function on x_train and x_test
42 print(x_train.describe())
43 print(x_test.describe())

```

12. Captured the outcome.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] X
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> && C:\Users\CSIS-Admin\Documents\CSIS672-Hashan\.venv\Scripts\python.exe c:/Users/CSIS-Admin/Documents/CSIS672-Hashan/hashan_ml.py

protocol_type service_flag land logged_in ... dst_host_error_rate dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate category
0 tcp ftp_data SF 0 0 ... 0.00 0.00 0.00 0.00 0.00 benign
1 udp other SF 0 0 ... 0.00 0.00 0.00 0.00 0.00 benign
2 tcp private SO 0 0 ... 1.00 1.00 0.00 0.00 dos
3 tcp http SF 0 1 ... 0.03 0.01 0.00 0.01 benign
4 tcp http SF 0 1 ... 0.00 0.00 0.00 0.00 benign

[5 rows x 42 columns]
   land logged_in is_host_login is_guest_login duration src_bytes dst_bytes ... flag_RSTR flag_S0 flag_S1 flag_S2 flag_S3 flag_SF flag_SH
0 0 0 0 0 0 491 0 ... False False False False False True False
1 0 0 0 0 0 146 0 ... False False False False False True False
2 0 0 0 0 0 0 0 ... False True False False False False False

[3 rows x 119 columns]
      land logged_in is_host_login ... dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate
count 125973.000000 125973.000000 125973.000000 ... 125973.000000 125973.000000 125973.000000
mean 0.000198 0.395736 0.000008 ... 0.278485 0.118832 0.120240
std 0.014086 0.489010 0.002817 ... 0.445669 0.306557 0.319459
min 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
25% 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
50% 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
75% 0.000000 1.000000 0.000000 ... 1.000000 0.000000 0.000000
max 1.000000 1.000000 1.000000 ... 1.000000 1.000000 1.000000

[8 rows x 38 columns]
      land logged_in is_host_login ... dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate
count 22543.000000 22543.000000 22543.000000 ... 22543.000000 22543.000000 22543.000000
mean 0.000311 0.442222 0.000488 ... 0.099430 0.233351 0.226649
std 0.017619 0.496661 0.022085 ... 0.281872 0.387204 0.400851
min 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
25% 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
50% 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000
75% 0.000000 1.000000 0.000000 ... 0.000000 0.360000 0.170000
max 1.000000 1.000000 1.000000 ... 1.000000 1.000000 1.000000

[8 rows x 38 columns]
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>

```

13. Ran the minmax scaler on the X features. First initialize a minmax object and use the object to both fit and transform the x, training data. For the x test data, only implement, transform using following commands.

```
45 #Run the minmax scaler on the X features
46 scaler = MinMaxScaler()
47
48 d_x_train = pd.DataFrame(x_train)
49 d_x_train[cont_features] = scaler.fit_transform(d_x_train[cont_features])
50
51 d_x_test = pd.DataFrame(x_test)
52 d_x_test[cont_features] = scaler.transform(d_x_test[cont_features])
53
54 print(d_x_train.head())
55 print(d_x_test.head())
```

Outcome from above commands.

```
[5 rows x 119 columns]
```

	land	logged_in	is_host_login	is_guest_login	duration	src_bytes	dst_bytes	...	flag_RST	flag_S0	flag_S1	flag_S2	flag_S3	flag_SF	flag_SH
0	0.0	0	0	0	0.0	3.558064e-07	0.000000e+00	...	False	False	False	False	False	True	False
1	0.0	0	0	0	0.0	1.057999e-07	0.000000e+00	...	False	False	False	False	False	True	False
2	0.0	0	0	0	0.0	0.000000e+00	0.000000e+00	...	False	True	False	False	False	False	False
3	0.0	1	0	0	0.0	1.681203e-07	6.223962e-06	...	False	False	False	False	False	True	False
4	0.0	1	0	0	0.0	1.442067e-07	3.206260e-07	...	False	False	False	False	False	True	False

```
[5 rows x 119 columns]
```

	land	logged_in	is_host_login	is_guest_login	duration	src_bytes	dst_bytes	...	flag_RST	flag_S0	flag_S1	flag_S2	flag_S3	flag_SF	flag_SH
125974	0.0	0	0	0	0.000000	0.000000e+00	0.000000e+00	...	False	False	False	False	False	False	False
125975	0.0	0	0	0	0.000047	9.408217e-06	0.000000e+00	...	False	False	False	False	False	True	False
125976	0.0	0	0	0	0.000000	1.449313e-08	0.000000e+00	...	False	False	False	False	False	True	False
125977	0.0	0	0	0	0.000023	0.000000e+00	1.145093e-08	...	False	False	False	False	False	True	False
125978	0.0	1	0	0	0.000000	1.934833e-07	1.108068e-05	...	False	False	False	False	False	True	False

```
[5 rows x 119 columns]
```

```
o (.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>
```


14. Separated the target feature, *y*, into training and testing portions with corresponding lengths to the predictor features training and testing datasets. Then printed the head for both commands and printed out the unique values in the dataset *y_total*.

```
57 #Seperate Target y for Train and Test
58 y_train = y_total[0:125973]
59 y_test = y_total[125974:148517]
60
61 print(pd.DataFrame(y_train).head())
62 print(pd.DataFrame(y_test).head())
63 # Print all unique attack categories
64 print(y_total.unique())
65
```

Output

```
[5 rows x 119 columns]
category
0    benign
1    benign
2      dos
3    benign
4    benign
category
125974    dos
125975    benign
125976    probe
125977    probe
125978    benign
['benign' 'dos' 'r21' 'probe' 'ur2']
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>
```

15. To train a classifier to identify an attack category based on the values of predictor features, *X*. Attack categories are *benign*, *dos*, *probe*, *r21*, *u2r*. These should be similar to the unique values in the target, *Y*, founded in the preceding step.

Created a classifier object, then used the values in training data, *x* and *y* to fit a model.

After fitting the model, I used it to predict target values based on the test subset of data, *d_x_test*

Then, generated a confusion matrix and printed it.

```
67 #Train the Classifier
68 #Classifier Objects
69 classifier = DecisionTreeClassifier(random_state=0)
70 classifier.fit(d_x_train, y_train)
71
72 #Predict target values on test set
73 pred_y = classifier.predict(d_x_test)
74
75 #Generate a confusion matrix and print it
76 results = confusion_matrix(y_test, pred_y)
77
78 print(results)
79
```

Outcome

```
[[9358  59  290   3   1]
 [1543 6000   92   0   0]
 [ 671  211 1541   0   0]
 [2048   4   17  160 345]
 [ 177   0   3   6  14]]
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>
```

16. Generated an error for the model and printed it.

```
80 #Generate the error and print
81 error = zero_one_loss(y_test, pred_y)
82
83 print(error)
```

Output

```
0.24264738499756022
```

```
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> 
```

After we trained the Decision Tree Classifier, the data sets detect various network attack categories, including benign, DoS (denial of service), R2L (remote-to-local), probe, and U2R (user-to-root) attacks. The matrix showed 9358 correctly classified benign samples and 290 mislabeled as R2L, 6000 correctly detected DoS instances and 1543 mistakes as Benign, 1541 correct R2L detections and 671 confused with Benign, 160 correct Probe detections and 2048 misclassified as Benign, 14 correct U2R detections and many others misclassified as Benign.

This means the diagonal values (9358, 6000, 1541, 160, 14) represent correct predictions and the off-diagonal values show misclassifications.

Error means incorrect prediction rate. In this lab the error rate indicates as 0.24264738499756022, meaning about 24.26 % of total predictions were incorrect. The accuracy of prediction is 75.74 %.

$$\begin{aligned}\text{Accuracy} &= 1 - \text{Error Rate} \\ &= 1 - 0.2426 \\ &= 0.7574 = 75.74\%\end{aligned}$$

Part 1- All commands

```

hashan_ml.py X
hashan_ml.py > ...
1  # Import the necessary packages
2  import pandas as pd
3  import numpy as np
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.compose import ColumnTransformer, make_column_selector
6  from sklearn.preprocessing import MinMaxScaler
7  from sklearn.tree import DecisionTreeClassifier
8  from sklearn.metrics import confusion_matrix, zero_one_loss
9
10 # Load the Data
11 hashan_data_all = pd.read_csv("C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/lab_data/lab_data_kdd_c.csv")
12
13 #Print the head of the data
14 print(hashan_data_all.head())
15
16 #Separate Predictors and Target
17 x_total = hashan_data_all.iloc[:, :-1]
18 y_total = hashan_data_all ['category']
19
20 #Identify and seperate Feature Types
21 cont_features = ['duration', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',
22                 'num_compromised', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files',
23                 'num_outbound_cmds', 'count', 'srv_count', 'error_rate', 'srv_error_rate', 'rerror_rate', 'srv_rerror_rate',
24                 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
25                 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
26                 'dst_host_error_rate', 'dst_host_srv_error_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate']
27
28 binary_features = ['land', 'logged_in', 'root_shell', 'su_attempted', 'is_host_login', 'is_guest_login']
29 nominal_features = ['protocol_type', 'service', 'flag']
30
31 #Convert nominal features into indicator features
32 all_x_data = pd.get_dummies(x_total, columns= nominal_features, drop_first=True)
33
34 #Check the outcome of the conversion
35 print(all_x_data[0:3])
36
37 #Seperate Dataset into Training and Testing
38 x_train = all_x_data.iloc[0:125973]
39 x_test = all_x_data.iloc[125974:148517]
40
41 #Run the describe() function on x_train and x_test
42 print(x_train.describe())
43 print(x_test.describe())
44
45 #Run the minmax scaler on the X features
46 scaler = MinMaxScaler()
47
48 d_x_train =pd.DataFrame(x_train)
49 d_x_train[cont_features] = scaler.fit_transform(d_x_train[cont_features])
50

```

```

51 d_x_test = pd.DataFrame(x_test)
52 d_x_test[cont_features] = scaler.transform(d_x_test[cont_features])
53
54 print(d_x_train.head())
55 print(d_x_test.head())
56
57 #Seperate Target y for Train and Test
58 y_train = y_total[0:125973]
59 y_test = y_total[125974:148517]
60
61 print(pd.DataFrame(y_train).head())
62 print(pd.DataFrame(y_test).head())
63
64 # Print all unique attack categories
65 print(y_total.unique())
66
67 #Train the Classifier
68 #Classifier Objects
69 classifier = DecisionTreeClassifier(random_state=0)
70 classifier.fit(d_x_train, y_train)
71
72 #Predict target values on test set
73 pred_y = classifier.predict(d_x_test)
74
75 #Generate a confusion matrix and print it
76 results = confusion_matrix(y_test, pred_y)
77
78 print(results)
79
80 #Generate the error and print
81 error = zero_one_loss(y_test, pred_y)
82
83 print(error)
84

```

Part 1- All Outputs

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> & C:\Users\CSIS-Admin\Documents\CSIS672-Hashan\.venv\Scripts\python.exe c:/Users/CSIS-Admin/Documents/CSIS672-Hashan/hashan_ml.py
protocol_type service flag land logged_in ... dst_host_error_rate dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate category
0 tcp ftp_data SF 0 0 ... 0.00 0.00 0.00 0.00 benign
1 udp other SF 0 0 ... 0.00 0.00 0.00 0.00 benign
2 tcp private S0 0 0 ... 1.00 0.00 0.00 0.00 dos
3 tcp http SF 0 1 ... 0.03 0.01 0.00 0.01 benign
4 tcp http SF 0 1 ... 0.00 0.00 0.00 0.00 benign

[5 rows x 42 columns]
 land logged_in is_host_login is_guest_login duration src_bytes dst_bytes ... flag_RST flag_S0 flag_S1 flag_S2 flag_S3 flag_SF flag_SH
0 0 0 0 0 0 491 0 ... False False False False False True False
1 0 0 0 0 0 146 0 ... False False False False False True False
2 0 0 0 0 0 0 0 ... False True False False False False False

[3 rows x 119 columns]
 land logged_in is_host_login is_guest_login ... dst_host_error_rate dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate
count 125973.000000 125973.000000 125973.000000 125973.000000 ... 125973.000000 125973.000000 125973.000000 125973.000000
mean 0.000198 0.395736 0.000000 0.009423 ... 0.284452 0.278485 0.118832 0.120240
std 0.014086 0.489010 0.002817 0.096612 ... 0.444784 0.445669 0.306557 0.319459
min 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
25% 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
50% 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
75% 0.000000 1.000000 0.000000 0.000000 ... 1.000000 1.000000 0.000000 0.000000
max 1.000000 1.000000 1.000000 1.000000 ... 1.000000 1.000000 1.000000 1.000000

[8 rows x 38 columns]
 land logged_in is_host_login is_guest_login ... dst_host_error_rate dst_host_srv_error_rate dst_host_rerror_rate dst_host_srv_rerror_rate
count 22543.000000 22543.000000 22543.000000 22543.000000 ... 22543.000000 22543.000000 22543.000000 22543.000000
mean 0.000311 0.442222 0.000488 0.028435 ... 0.097818 0.099430 0.233351 0.226649
std 0.017619 0.496661 0.022885 0.166214 ... 0.273144 0.281872 0.387204 0.400851
min 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
25% 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
50% 0.000000 0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
75% 0.000000 1.000000 0.000000 0.000000 ... 0.000000 0.000000 0.360000 0.170000
max 1.000000 1.000000 1.000000 1.000000 ... 1.000000 1.000000 1.000000 1.000000

[8 rows x 38 columns]

```

```

land logged_in is_host_login is_guest_login duration src_bytes dst_bytes ... flag_RST flag_S0 flag_S1 flag_S2 flag_S3 flag_SF flag_SH
0 0.0 0 0 0 0.0 3.558064e-07 0.000000e+00 ... False False False False False True False
1 0.0 0 0 0 0.0 1.057999e-07 0.000000e+00 ... False False False False False True False
2 0.0 0 0 0 0.0 0.000000e+00 0.000000e+00 ... False True False False False False False
3 0.0 1 0 0 0.0 1.681203e-07 6.223962e-06 ... False False False False False True False
4 0.0 1 0 0 0.0 1.442067e-07 3.206260e-07 ... False False False False False True False

[5 rows x 119 columns]
land logged_in is_host_login is_guest_login duration src_bytes dst_bytes ... flag_RST flag_S0 flag_S1 flag_S2 flag_S3 flag_SF flag_SH
125974 0.0 0 0 0 0.000000 0.000000e+00 0.000000e+00 ... False False False False False False False
125975 0.0 0 0 0 0.000047 9.408217e-06 0.000000e+00 ... False False False False False True False
125976 0.0 0 0 0 0.000000 1.449313e-08 0.000000e+00 ... False False False False False True False
125977 0.0 0 0 0 0.000023 0.000000e+00 1.145093e-08 ... False False False False False False False
125978 0.0 1 0 0 0.000000 1.934833e-07 1.108068e-05 ... False False False False False True False

[5 rows x 119 columns]
category
0 benign
1 benign
2 dos
3 benign
4 benign
category
125974 dos
125975 benign
125976 probe
125977 probe
125978 benign
['benign' 'dos' 'r21' 'probe' 'ur2']
[[9358 59 290 3 1]
 [1543 6000 92 0 0]
 [ 671 211 1541 0 0]
 [2048 4 17 160 345]
 [ 177 0 3 6 14]]
0.24264738499756022
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>

```

Part 2 - K-Nearest Neighbor

1. Used the k-nearest neighbor classifier on the same dataset.

```
85 #Part 2 - k-nearest neighbor classifier
86 from sklearn.svm import LinearSVC
87 from sklearn.metrics import confusion_matrix
88
89 classifier = LinearSVC()
90 classifier.fit(d_x_train, y_train)
91 pred_y = classifier.predict(d_x_test)
92
93 results = confusion_matrix(y_test, pred_y)
94 print(results)
95
96 error = zero_one_loss(y_test, pred_y)
97 print(error)
```

Output

```
[[9008  88  612   2   1]
 [1546 6075  14   0   0]
 [ 703  136 1497  87   0]
 [2502   2   1  69   0]
 [ 187   0   0   3  10]]
0.2610122876280886
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>
```

k-nearest neighbor classification model detects various network attack categories, including benign, DoS (denial of service), R2L (remote-to-local), probe, and U2R (user-to-root) attacks. The matrix showed 9008 correctly classified benign samples and 612 mislabeled as R2L, 6075 correctly detected DoS instances and 1546 mistakes as Benign, 1497 correct R2L detections and 703 confused with Benign, 69 correct Probe detections and 2502 misclassified as Benign, 10 correct U2R detections and many others are misclassified as Benign.

This means the diagonal values (9008, 6075, 1497, 69, 10) represent correct predictions and the off-diagonal values show misclassifications.

Error means incorrect prediction rate. In this lab the error rate indicates as 0.2610122876280886, meaning about 26.10 % of total predictions were incorrect. The accuracy of prediction is 73.90 %.

$$\begin{aligned}\text{Accuracy} &= 1 - \text{Error Rate} \\ &= 1 - 0.2610 \\ &= 0.7390 = 73.90\%\end{aligned}$$

Part 3 – Comparison

Both decision tree and k-nearest neighbor (KNN) classification models were implemented to detect multiple network attack categories including Benign, DoS (denial of service), R2L (remote-to-local), probe, and U2R (user-to-root) attacks. Confusion matrix and the error rate metric were used by each model.

According to the results given by each model, the decision tree classifier achieved 75.74% accuracy and KNN classifier achieved 73.90% accuracy. Sametime, the decision tree classifier had 0.2426 error rate and KNN classifier had 0.2610 error rate. The decision tree classifier showed slightly higher accuracy and lower error when compared to k-nearest neighbor.

Conclusion

This lab successfully established a fully functional Python development environment designed for data analysis using machine learning applications. The decision tree classifier produced an accuracy of 75.74 % while the k-nearest neighbor model achieved 73.90 %. Both models show the effectiveness of detecting common network attack categories including Benign and DoS (denial of service). But when it comes to R2L (remote-to-local), probe, and U2R (user-to-root) attacks were difficult to identify using these two modules.

This lab demonstrates that while machine-learning models offer a good approach for intrusion detection, their deployment in cybersecurity demands careful design, realistic performance evaluation, and an awareness of security-specific challenges such as adversarial attacks, and data imbalance.

References

1. Chio, C., & Freeman, D. (2018). *Machine Learning and Security: Protecting Systems with Data and Algorithms*. O'Reilly Media.
2. GeeksforGeeks. (2025, July 11). *Pandas Read CSV in Python*. Sanchhaya Education Private Limited. https://www.geeksforgeeks.org/pandas/python-read-csv-using-pandas-read_csv/
3. Microsoft Docs. (2025). *Python in Visual Studio Code*. Microsoft. Retrieved October 28, 2025, from <https://code.visualstudio.com/docs/languages/python>
4. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media
5. Minnesota State University Moorhead. (2025). Course content: Cybersecurity Analytics. D2L Brightspace. Retrieved November 17, 2025 from <https://mnstate.learn.minnstate.edu/d2l/le/content/7131515/viewContent/75421588/View>